



MÓDULO 1 · AULA 09

Aula 09

100% de cobertura $\neq$ 100% testado

Testes unitários, cobertura de código, teste de mutação — e por que aquele número verde não promete software sem bug.

Instrutor: Douglas Adriano Queiroz

O que vamos ver hoje



Teste unitário

A menor peça, isolada



Cobertura

O que o número mede



Ferramentas

Jest, Istanbul e Stryker



Teste de mutação

Como funciona e como configurar



O ponto cego

O código que não existe



Robustez e CTFL

O que ajustar no código

Teste unitário: a menor peça, isolada

Testa uma unidade sozinha — uma função, um módulo — sem banco, sem rede e sem tela. Roda em milissegundos e aponta o dedo para a linha exata que quebrou.

Rápido: centenas de casos em segundos, a cada vez que você salva.

Isolado: quando falha, você já sabe onde procurar.

Barato: é a base da pirâmide — o andar mais largo e mais barato de manter.

Limitado: não enxerga integração, ambiente, nem o usuário de verdade.



Analogia da casa

É testar cada tomada com a chave de teste antes de fechar a parede. Rápido e barato. Mas nenhuma tomada aprovada garante que o chuveiro não vai desarmar o disjuntor quando tudo ligar junto.



Leve daqui: Teste unitário é a fundação da casa — não é a casa inteira.

Cobertura de código: o que o número mede

Cobertura é a fração do código que os seus testes executaram. A ferramenta marca cada linha visitada durante a rodada e devolve um percentual.

Mede: quais linhas, ramos e funções o teste percorreu.

Não mede: se você verificou o resultado ou só executou a linha.

Não mede: quais valores entraram na função.

Não mede: o requisito que ninguém chegou a implementar.



Analogia da casa

É a planta da casa com os cômodos que o inspetor visitou pintados de verde. Ele passou por todos — mas não disse se abriu as torneiras, nem se falta um banheiro que estava no projeto.



Leve daqui: Cobertura diz por onde o teste passou. Nunca diz se o software está correto.

O que a cobertura enxerga e o que ela não vê



Ela enxerga

- Linha ou comando executado pelo menos uma vez.
- Decisão avaliada como verdadeira e como falsa.
- Função ou método que chegou a ser chamado.
- Código que nenhum teste alcança — as linhas vermelhas do relatório.



Ela é cega para

- Se o teste verificou o resultado ou só passou por ali.
- Quais valores entraram: 0, negativo, null, NaN, string vazia.
- Requisito que nunca virou código: zero linha, zero descoberto.
- Integração, ambiente, desempenho, acessibilidade e o usuário real.



Leve daqui: Cobertura alta não é prova. Cobertura baixa é denúncia — e essa parte vale ouro.

Nem todo 100% é o mesmo 100%



Comando (linha)

Cada linha rodou ao menos uma vez. É o critério mais fraco — e o mais citado.



Decisão (ramo)

Cada if avaliado como verdadeiro E como falso. Já é bem melhor que linha.



Condição

Cada condição atômica de um && ou || avaliada dos dois jeitos.



MC/DC

Cada condição precisa mudar o resultado sozinha. Exigido em software aeronáutico.



Caminho

Todas as combinações de caminho. Inviável fora de funções minúsculas.



Mutação

Não é cobertura: altera o código e vê se algum teste reclama. Mede a força da suíte.

As ferramentas que usamos no projeto



Node.js 22

Roda o JavaScript fora do navegador. Código CommonJS puro, sem bundler.



Jest 29

Framework de teste unitário: describe, it, expect, it.each, jest.fn() e rejects.



Istanbul

Motor de cobertura embutido no Jest. Instrumenta o código e gera o relatório HTML.



Stryker 8

Teste de mutação em JavaScript. Altera o código e mede o que a suíte detecta.



jest-runner

Plugin que faz o Stryker rodar a suíte Jest contra cada mutante gerado.



Em outras stacks

PIT (Java), Stryker.NET (C#), mutmut ou Cosmic Ray (Python), go-mutesting (Go).

O experimento: rode você mesmo



npm install

Prepara o projeto



npm run cobertura

100% nos 4 critérios



npm run realidade

32 testes expõem os bugs



npm run mutacao

20 mutantes sobrevivem

O projeto cobertura-garapuvu

Nove arquivos JavaScript, todos com defeito, e uma suíte de 20 testes que fica 100% verde em cima deles. Está no repositório do curso, na pasta cobertura-garapuvu/. Rode antes de acreditar em mim.

Onde o bug se escondeu



Tipo inesperado

string, null, undefined, array e objeto entram e saem resultados errados, sem erro.



Condição composta

|| no lugar de &&: 100% de linha e de ramo, e a regra continua errada.



Valor limite

> no lugar de >=. Testar 300 e 120 não acha o defeito que mora exatamente em 240.



Asserção fraca

toBeDefined() executa a linha e não verifica nada. É teste sem oráculo.



Efeito colateral

sort() sem comparador ordena como texto e ainda muda o array de quem chamou.



Requisito ausente

A regra que nunca virou código. Zero linha escrita = 100% de cobertura.

Como funciona o teste de mutação



A pergunta que muda tudo

Cobertura pergunta "o teste passou por essa linha?". Mutação pergunta "se eu quebrar essa linha de propósito, algum teste reclama?". O seu código-fonte nunca é alterado: a ferramenta trabalha sobre uma cópia.

Os estados de um mutante



Killed (morto)

Pelo menos um teste falhou. A suíte percebeu a alteração. É o que você quer.



Survived

Todos os testes passaram mesmo assim. Buraco real: ninguém verifica esse comportamento.



Timeout

O mutante travou, por exemplo num laço infinito. Conta como detectado.



No coverage

Nenhum teste sequer executou aquela linha. É buraco de cobertura, antes de asserção.



Runtime error

O mutante nem chegou a rodar. Sai da conta, não entra no denominador.



0 score

$\text{score} = (\text{mortos} + \text{timeouts}) \div \text{mutantes válidos} \times 100$. Aqui: $(206 + 1) \div 210 = 98,57\%$.

A configuração do Stryker no projeto

O arquivo de configuração é curto, mas cada chave tem uma consequência prática. São quatro que valem entender antes de rodar em projeto de verdade.

mutate: quais arquivos são alterados. Aqui, src/corrigido/*.js.

testMatch: quais suítes julgam os mutantes. É independente do mutate.

coverageAnalysis: "perTest" roda só os testes que tocam a linha mutada. Sem isso, fica lento demais.

thresholds.break: null não derruba o build; 80 vira portão de CI. Pense duas vezes.



Duas chaves independentes

É essa separação que permite o experimento da aula: o mesmo código julgado por suítes diferentes. A suíte fraca dá 79,59%; a suíte forte dá 98,57%. Mesmo código, mesma cobertura de 100%.



Leve daqui: Configurar mutação é escolher o que mutar e quem julga. O resto é detalhe.

Nem todo sobrevivente é buraco de teste



Mutante sobrevivente

- A alteração mudou o comportamento e nenhum teste reclamou.
- É buraco real: falta assertão ou falta caso de teste.
- Pode também denunciar código redundante — condição que não faz diferença nenhuma.
- Ação: escrever o teste que mata, ou simplificar o código.



Mutante equivalente

- A alteração NÃO muda o comportamento observável para nenhuma entrada.
- Exemplo real daqui: remover `.trim()` quando já existe `.filter(Boolean)`.
- Detectá-los automaticamente é indecidível — equivale ao problema da parada.
- Ação: registrar e seguir em frente. Não dá para matar.



Leve daqui: 100% de score de mutação quase nunca é atingível. Perseguir-lo é desperdício.

E em linguagens mais tipadas?



O que muda a favor

- O compilador já barra "string onde se esperava número": o Caso 01 nem existiria.
- Não faz sentido mutar o tipo de um valor — sobram mutantes de valor e de condição.
- Mutante inválido sai da conta: no Stryker+TS vira CompileError; o PIT muta bytecode já checado.
- Mutadores como NULL_RETURNS morrem por NullPointerException, às vezes sem asserção nenhuma.



O que NÃO muda

- Nas fronteiras o defeito volta: JSON.parse devolve any, "as Foo" some em runtime, any desliga tudo.
- Em Java e C#: desserialização, reflection, casts e type erasure quebram em runtime igual.
- Nem Stryker nem PIT têm mutador de throw/try/catch — o efeito é sempre indireto.
- Não existe estudo comparando eficácia em tipada × dinâmica. Quem afirmar, está opinando.



Leve daqui: Tipagem cobre o miolo do código, não as bordas. Guarda de contrato continua obrigatória.

O placar do experimento



Suíte fraca: 20 testes

- Cobertura: 100% em comandos, ramos, funções e linhas.
- coverageThreshold de 100% configurado — e aprovado no CI.
- Score de mutação: 79,59%. Dos 98 mutantes, 20 sobreviveram.
- Defeitos encontrados: nenhum.



Suíte forte: 122 testes

- Cobertura: 100% nos mesmos quatro critérios. Empate.
- Score de mutação: 98,57%. Dos 210 mutantes, 3 sobreviveram.
- Os 3 que restaram são equivalentes — não dá para matar.
- Defeitos encontrados: todos, mais um que ninguém suspeitava.



Leve daqui: A cobertura empata as duas colunas. Só a mutação mostra a diferença.

O que ajustar no código para ficar robusto



Guarda de contrato

Valide tipo e faixa no topo e lance `TypeError`.
Nunca devolva `undefined` em silêncio.



Sem redundância

Se `Number.isFinite` já resolve, tire o `typeof`. Ramo a mais é teste a mais para manter.



Erro específico

Mensagem exata e classe nomeada:
`VoluntarioNaoEncontradoError`,
`TurmaLotadaError`.



Função pura

Copie antes de ordenar: `[...lista].sort()`. Nunca acumule em variável de módulo.



Zero, um e muitos

Trate a coleção vazia antes de dividir. São três casos de teste, sempre.



Dinheiro em centavos

`Math.round(Number((v*100).toFixed(6)))`. E decida se arredonda por item ou no total.

Como escrever o teste que mata o mutante



$x \geq 240$ vira $x > 240$

Teste exatamente 239, 240 e 241. A fronteira, nunca o meio da faixa.



$a \&\& b$ vira $a \parallel b$

Teste cada operando isolado. Monte a tabela-verdade completa.



if (cond) vira if (true)

Teste o caminho em que a condição é falsa E o resultado muda.



'mensagem' vira vazio

Assertão com `.toThrow("mensagem exata")`, não com `toThrow()` vazio.



$a + b$ vira $a - b$

Use valores em que soma e subtração dão diferente. Evite $0 + 0$.



[...lista] vira []

Compare o array inteiro com `toEqual([...])`, em ordem, e confira o tamanho.

Caso 01: o clássico do JavaScript

A função calcula o valor da inscrição com desconto. O teste passa 200 e 10, recebe 180, e a cobertura fecha em 100%. Ninguém testou o que o formulário manda de verdade.

`calcularInscricao('200', 0)`: devolve a string '200' — o front concatena depois.

`calcularInscricao(null, 10)`: devolve 0 — inscrição de graça.

`calcularInscricao(undefined, 10)`: devolve NaN — "R\$ NaN" na tela do aluno.

`calcularInscricao(200, 150)`: devolve -100 — o projeto paga o aluno.



Analogia da casa

É a fechadura que aceita qualquer chave torta e gira mesmo assim. Não trava, não avisa: só abre a porta errada. Em JavaScript, tipo errado quase nunca vira exceção — vira resultado errado em silêncio.



Leve daqui: Teste null, undefined, string, array, negativo, zero e valor gigante. Sempre.

Caso 09: o código que não existe

A regra dizia: só aceitar inscrição se houver vaga na turma. A validação nunca foi escrita — e linha que não existe não fica descoberta.

Nenhuma ferramenta acha: cobertura só mede o código que existe.

Quem acha: revisão de requisitos e teste estático (CTFL 4.0, capítulo 3).

Quem acha: rastreabilidade requisito → caso de teste (CTFL 4.0, seção 1.4.4).

Quem acha: o teste exploratório de quem conhece o negócio.



Analogia da casa

O inspetor visitou todos os cômodos e assinou o laudo. Só que a planta previa um banheiro que o pedreiro esqueceu de construir — e cômodo que não existe não entra na lista de cômodos não visitados.



Leve daqui: O furo mais caro é sempre o requisito que ninguém chegou a escrever.

Onde isso encosta na CTFL 4.0



1.3 Princípios

"Testes mostram a presença de defeitos, não a sua ausência." É o Caso 09 inteiro.



1.4.4 Rastreabilidade

Cobertura deve ser medida contra a base de teste, não só contra o código.



Cap. 3 Teste estático

Revisão acha defeito em requisito, antes de existir código para executar.



4.2 Caixa-preta

Equivalência (4.2.1), valor limite (4.2.2) e tabela de decisão (4.2.3).



4.3 Caixa-branca

Cobertura de comandos (4.3.1) e de ramos (4.3.2). E 4.3.3: por que ela não basta.



Fora da Foundation

Teste de mutação é conteúdo do nível Advanced. Está aqui porque o mercado usa.

Quando isso custou caro



Ariane 5 · 1996

- O foguete se destruiu cerca de 37 segundos após o início do lançamento.
- O software vinha do Ariane 4, onde era testado e aprovado havia anos.
- Um número de 64 bits não coube em 16 bits e derrubou as duas unidades de referência inercial.
- Aquele trecho nunca tinha sido exercitado com os dados de voo do foguete novo.



Knight Capital · 2012

- Cerca de US\$ 440 milhões perdidos em 45 minutos de pregão.
- Uma implantação reativou por engano um trecho de código antigo e desativado.
- Nenhum teste cobria aquele caminho na configuração nova.
- A empresa precisou de socorro financeiro em dias e foi vendida meses depois.



Leve daqui: Nos dois casos o software "passava nos testes". Faltou testar com os dados e o ambiente reais.



Resumão da aula



Cobertura mede quais linhas o teste executou, não se o resultado foi verificado.



Teste de mutação altera o código de propósito e mede a força da suíte, não o tamanho dela.



Ferramentas: Jest + Istanbul para cobertura, Stryker para mutação. Tudo com um npm install.



Mutante sobrevivente é buraco de teste ou código redundante. Equivalente não dá para matar.



Em linguagem tipada o compilador cobre o miolo, mas não as bordas: guarda de contrato continua.



100% de cobertura não é ausência de bug — testes mostram a presença de defeitos (CTFL 1.3).



Glossário da aula

Teste unitário — Testa a menor parte isolada do software

Cobertura de código — % do código executado durante os testes

Cobertura de comando — Cada linha executada ao menos uma vez

Cobertura de ramo — Cada decisão avaliada como verdadeira e falsa

MC/DC — Cada condição influencia o resultado sozinha

Oráculo de teste — A fonte que diz qual é o resultado correto

Mutante — Versão do código com uma alteração proposital

Mutante morto — Alteração que fez algum teste falhar

Mutante sobrevivente — Alteração que nenhum teste detectou

Mutante equivalente — Alteração que não muda o comportamento



Referências e para aprofundar

BSTQB / ISTQB. Syllabus CTFL v4.0 — 1.3 (princípios), 1.4.4 (rastreabilidade), 4.2 e 4.3 (técnicas).

Jest. Framework de teste e motor de cobertura — jestjs.io/docs/configuration

Stryker Mutator. Mutadores, estados do mutante e cálculo do score — stryker-mutator.io

PIT. Teste de mutação em Java, lista de mutadores — pitest.org/quickstart/mutators

Para praticar: o projeto cobertura-garapuvu/ no repositório, com README e exercícios.